

# CDFD Runtime Paper 05: CDFL and Executable Model Grammar

Steve Bico Mujjabi, MD

Independent Researcher

ORCID: 0009-0001-0556-5516

May 2026

## Abstract

**Executive synthesis.** CDFL is the command language of the CDFD Runtime. It lets a user define systems, assign domains, run simulations, observe metrics, and trigger rules without writing Python directly. This paper explains the compiler path and the CLI contract that now makes CDFL reproducible through `cdfd.py validate`, `cdfd.py run`, and JSON export.

### Runtime evidence path

Prior CDFD mathematics  $\longrightarrow$  CDFL/runtime state  $\longrightarrow$  bounded output  $\longrightarrow$  falsification target.

Figure 1: Conceptual schematic for the runtime papers. The engine translates the mathematical frame from the prior CDFD papers into executable CDFL/runtime diagnostics; candidate outputs remain tied to explicit tests before any stronger claim is made.

## 1 Prior CDFD Basis

This manuscript does not rederive the mathematical core of CDFD. It uses the Mujjabi Adaptive Operating Ratio and the constrained-vacuum notation introduced in Part I [1], the torus-knot hierarchy and boundary-mode work from the Part I sequence [2, 3], the public balance law developed in the Part I capstone [4], the Part I transport tests [5], and the Life Number and tri-regime biology bridge from Part II [6]. The runtime papers therefore act as an execution layer: they keep the mathematics connected to commands, outputs, falsification tests, and domain-specific limits.

## 2 Why a Language Is Needed

Directly manipulating NumPy arrays is powerful but fragile. A scientific runtime needs a stable way to describe a model, run it, and save the output so that the same model can be reviewed by another reader. CDFL provides that grammar.

## 3 Compiler Pipeline

### 3.1 Tokenization

`dsl/lexer.py` and `dsl/tokens.py` convert source text into tokens. The language recognizes keywords such as `SET`, `SYSTEM`, `RULE`, `RUN`, `OBSERVE`, and `PATIENT`.

### 3.2 Parsing

`dsl/parser.py` converts tokens into AST nodes. A `SystemNode` stores the relation between a named system, a flow expression, and a constraint expression. A `RuleNode` stores threshold logic.

### 3.3 Execution

`dsl/executor.py` walks the AST. It resolves available variables, computes  $\Psi_s = (\Phi/C)SM_s$ , routes actions through the dry-run action gateway, and invokes the kernel for `RUN Engine` blocks.

## 4 Minimal Example

The active example is `examples/heat_flow.cdf1`:

```
SET domain: physics
```

```
SYSTEM HeatChannel {  
    flux: 1.2  
    constraint: 0.9  
    state: psi  
}
```

```
RULE HeatOverload {  
    IF psi > 1.1  
    ACTION reduce_flux  
}
```

```
RUN Engine {  
    duration: 0.05  
    dt: 0.01  
}
```

```
OBSERVE {  
    metrics: [psi]  
}
```

This model can be checked and run with:

```
python cdfd.py validate examples/heat_flow.cdf1  
python cdfd.py run examples/heat_flow.cdf1 --out outputs/heat_flow_run.json
```

## 5 Output Semantics

CDFL execution produces structured result blocks:

- **system**: computed  $\Phi$ ,  $C$ , and  $\Psi_s$  for named systems;
- **rule**: triggered actions and dry-run gateway output;
- **run**: kernel time series;
- **observe**: requested metrics.

The CLI wraps these blocks in a runtime result envelope with provenance and a finite audit.

## 6 Connection to Discoveries

The discovery experiments currently use Python scripts, but CDFL is the durable experiment front door. A discovery candidate eventually has:

- a CDFL model file;
- a saved JSON/HDF5 output;
- a report entry;
- a falsification condition;
- a rerun command.

This is how candidate findings move from exploratory notebooks into reproducible runtime claims.

## 7 Re-Running the Discoveries in CDFL

The final development target is to rerun the selected discovery set from CDFL rather than from ad hoc Python scripts. The short path is not to rewrite every experiment by hand. The runtime needs a small CDFL extension for parameter sweeps, output manifests, and named test families:

```
DISCOVERY MujjabiVacuumHysteresis {  
  model: vacuum_memory  
  sweep: pulse_strength [1.0, 2.0, 4.0]  
  output: experiments/outputs/discovery_vacuum_hysteresis.h5  
  falsify: residual_vanishes_under_controls  
}
```

Once that grammar exists, the same discoveries can be run through `cdfd.py run`, exported as result envelopes, and displayed by VOS or CDFD Studio without changing the engine mathematics.

## 8 Limitations

The current CDFL grammar is intentionally small. It does not yet cover every experiment pattern in `experiments/notebooks`. The next development target is to make common sweeps, parameter grids, and discovery runs expressible in CDFL without losing Python-level control for advanced experiments.

## 9 Conclusion

CDFL turns CDFD from a codebase into an executable manuscript language. Its scientific value depends on reproducibility: validate the model, run it, save the output, and make the failure conditions explicit.

## References

- [1] Steve Bico Mujjabi, MD. *Vortex Stability in a Constrained Transport Vacuum and the Origin of the Fine-Structure Constant*. CDFD Part I Paper I, preprint, 2026.
- [2] Steve Bico Mujjabi, MD. *The Universal Torus Knot Hierarchy:  $Z_n$  Symmetry, the Cinquefoil Family, and the General Structure of CDFT Particle Families*. CDFD Part I Paper VI, preprint, 2026.
- [3] Steve Bico Mujjabi, MD. *Even- $n$  Torus Knots in CDFT: The Exclusion of  $T(2, 2)$ , the Extension to  $T(2, 2m)$ , and the Anti-Phase Mass Scaling Law*. CDFD Part I Paper IX, preprint, 2026.
- [4] Steve Bico Mujjabi, MD. *The Vacuum Equation of State: Deriving Torus Knot Self-Consistency from the Public CDFD Balance Law  $\Psi = \Phi/C$* . CDFD Part I Paper X, preprint, 2026.
- [5] Steve Bico Mujjabi, MD. *Friction, Superconductivity, Plasma States, and Transport Mysteries: Observable Tests of Adaptive Constraint*. CDFD Part I Paper XII, preprint, 2026.
- [6] Steve Bico Mujjabi, MD. *CDFD Part II: Origins of Life and Tri-Regime Bioenergetics*. Public release archive, 2026, <https://doi.org/10.5281/zenodo.20250821>.